



Society of Petroleum Engineers

SPE-227315-MS

Enhancing Reverse Time Migration Performance with Half-Precision on GPUs for Production Applications

Hongwei Liu, Strategic Modeling Technology, Advanced Research & Development, Saudi Aramco, Dhahran, Eastern Province, Saudi Arabia

Copyright 2025, Society of Petroleum Engineers DOI [10.2118/227315-MS](https://doi.org/10.2118/227315-MS)

This paper was prepared for presentation at the Middle East Oil, Gas and Geosciences Show (MEOS GEO), Manama, Bahrain, 16 – 18 September 2025.

This paper was selected for presentation by an SPE program committee following review of information contained in an abstract submitted by the author(s). Contents of the paper have not been reviewed by the Society of Petroleum Engineers and are subject to correction by the author(s). The material does not necessarily reflect any position of the Society of Petroleum Engineers, its officers, or members. Electronic reproduction, distribution, or storage of any part of this paper without the written consent of the Society of Petroleum Engineers is prohibited. Permission to reproduce in print is restricted to an abstract of not more than 300 words; illustrations may not be copied. The abstract must contain conspicuous acknowledgment of SPE copyright.

Abstract

Reverse Time Migration (RTM) is a computationally intensive process, often limited by memory bandwidth and storage I/O, particularly in large-scale production environments. In this work, we investigate the use of IEEE FP16 (half-precision floating-point) format to improve RTM performance on modern GPUs. Specifically, we store the wavefield in FP16 format in GPU memory and utilize FP16-based data transfers to CPU memory and disk, significantly reducing memory usage and I/O bandwidth without modifying the core computation in FP32. Synthetic and field data experiments demonstrate that this strategy introduces only negligible differences in migrated images compared to full-precision implementations, while achieving substantial gains in memory and disk efficiency. Further analysis explores the feasibility of using FP16 for computation, revealing that while performance improves, the numerical errors are too large for production-quality imaging. These results suggest that FP16 is highly effective for storage and communication in RTM workflows, but not yet suitable for core numerical operations in production-scale deployments.

Introduction

Reverse Time Migration (RTM) is one of the most accurate seismic imaging techniques available, capable of handling complex velocity models and imaging steep reflectors. Its full-wave equation modeling makes it an indispensable tool in both academic research and industrial exploration (Liu et al., 2013; Liu and Luo, 2020). However, this accuracy comes at a high computational cost. RTM typically requires storing the full forward wavefield for imaging condition evaluation during backward propagation, resulting in substantial memory, bandwidth, and disk I/O demands—especially in 3D and TTI applications (Liu et al., 2010; Liu et al., 2012).

To address these challenges, GPUs have become a widely adopted platform for RTM due to their high floating-point throughput and memory bandwidth (Liu et al., 2010; Micikevicius, 2009; Kronenberg et al., 2007; Moussa and Schuster, 2015). Over the past decade, GPU architectures have evolved rapidly from early programmable stages to highly parallel systems with dedicated hardware for mixed-precision operations. Multi-GPU strategies and workload mapping techniques have further enhanced scalability (Deng et al., 2021; Liu et al., 2019; Lin et al., 2022; Modave et al., 2015).

Despite these advances, memory remains a critical bottleneck, particularly for storing wavefields in RTM. One emerging strategy to alleviate memory pressure is to adopt reduced-precision formats such as FP16 (half precision). Originally developed to accelerate AI workloads, FP16 has demonstrated significant benefits in terms of memory footprint and data transfer efficiency (Ginsburg et al., 2020; NVIDIA, 2021; Micikevicius et al., 2018; Jia et al., 2019). Specialized hardware such as Tensor Cores and optimized software stacks in PyTorch and TensorFlow have made FP16 a mainstream solution in deep learning frameworks (Jouppi et al., 2017; PyTorch Team, 2020; Yu and Ma, 2021).

Inspired by these successes, recent studies have begun exploring the application of FP16 in high-performance computing and geophysical modeling. In seismic imaging, applications include compressing stored wavefields (Zhang et al., 2019; Zhu et al., 2021) and accelerating inversion workflows (Yin and Zhang, 2020). However, the suitability of FP16 for production-scale RTM remains an open question due to concerns over numerical stability and imaging accuracy.

In this paper, we investigate the application of FP16 to reduce GPU memory and disk I/O pressure in RTM workflows. We evaluate both synthetic and field data examples to validate that storing and transferring wavefields in FP16 format leads to negligible image quality degradation. We further assess the feasibility of performing RTM computation entirely in FP16, and demonstrate that while speed gains are notable, the resulting artifacts render this approach unsuitable for production use. The findings provide a practical guideline for leveraging reduced precision in production seismic imaging, balancing performance with accuracy.

Theory and Method

Reverse Time Migration Workflow and Computational Bottlenecks

Reverse Time Migration (RTM) reconstructs the seismic image by solving the two-way wave equation in both forward and backward directions. The process consists of three primary steps: (1) forward wavefield propagation from sources, (2) backward propagation of recorded data from receivers, and (3) imaging condition evaluation, typically performed as the zero-lag cross-correlation between the forward and backward wavefields at each time step.

Mathematically, RTM solves the second-order acoustic wave equation:

$$\frac{\partial^2 p(x, t)}{\partial t^2} = v^2(x) \nabla^2 p(x, t) + s(x, t), \quad (1)$$

where $p(x, t)$ is the pressure field, $v(x)$ is the velocity model, and $s(x, t)$ is the source term. The forward wavefield $p_f(x, t)$ is propagated and saved (either fully or in checkpoints), while the backward wavefield $p_b(x, t)$ is computed by reversing the source-time function and injecting the recorded data at receiver locations. The image $I(x)$ is constructed by:

$$I(x) = \int_0^T p_f(x, t) \cdot p_b(x, t) dt. \quad (2)$$

While the algorithm is conceptually straightforward, its implementation on realistic 3D models with fine grid spacing and long propagation times requires handling vast amounts of data. The primary bottlenecks in RTM arise from:

1. **Wavefield Storage:** For full cross-correlation imaging, the entire forward wavefield must be available during backward propagation. This can require tens to hundreds of gigabytes of memory for each shot in 3D RTM.
2. **Memory Bandwidth:** Even if enough GPU memory is available, transferring wavefields in and out of memory at each time step places extreme pressure on memory bandwidth and latency.

3. **Disk I/O Throughput:** In cases where memory is insufficient, wavefields are streamed to disk and reloaded, significantly slowing down the process and introducing I/O bottlenecks.
4. **Host-Device Transfers:** In hybrid CPU–GPU workflows, moving large volumes of wavefield data between host and device further contributes to performance degradation.

As RTM scales to increasingly large surveys and high-frequency content, these bottlenecks become more pronounced. Solutions such as optimal memory mapping strategies (Deng et al., 2021), GPU clustering (Modave et al., 2015; Lin et al., 2022), and wavefield reconstruction techniques (Liu et al., 2013) have been proposed, but even these cannot eliminate the fundamental limitations of full-precision wavefield storage. In this context, adopting reduced-precision representations, particularly FP16, emerges as a promising strategy for alleviating memory and I/O constraints without changing the underlying physics of wave propagation.

FP16 format in HPC

The FP16 (half-precision floating-point) format is a 16-bit numerical representation defined by the IEEE 754 standard, consisting of 1 sign bit, 5 exponent bits (with a bias of 15), and 10 fraction (mantissa) bits. This compact format significantly reduces memory usage and bandwidth compared to single-precision (FP32) and double-precision (FP64) representations, making it ideal for applications where extreme precision is not critical. FP16 has become especially important in the field of artificial intelligence (AI), where it enables faster and more energy-efficient training and inference, particularly in deep learning models. Modern GPUs, such as NVIDIA's Volta, Ampere, and Hopper architectures, include dedicated Tensor Cores that accelerate matrix operations using FP16 or mixed precision (e.g., FP16 input with FP32 accumulation), achieving performance gains of $4\times$ to $8\times$ over FP32 in AI workloads. Beyond AI, FP16 is gaining traction in high-performance computing (HPC) for storing intermediate results, reducing data movement, and enabling higher arithmetic throughput where exact precision is not mandatory—such as in preconditioning, visualization, or coarse physics stages. While limited in range and accuracy, FP16 holds substantial promise in hybrid HPC-AI workflows, where selective precision control can lead to both speed and efficiency gains.

As shown in Fig. 1, both FP32 and FP16 formats represent real numbers, and are widely used in numerical computing, with FP16 increasingly applied in AI and GPU-accelerated workloads due to its reduced memory and computational footprint. The primary difference between FP32 (single-precision) and FP16 (half-precision) floating-point formats lies in their bit-width, which directly impacts numerical range, precision, and hardware performance. Both formats follow the IEEE 754 standard and represent values in the form:

$$(-1)^{\text{sign}} \cdot 2^{\text{exponent-bias}} \cdot \left(1 + \frac{\text{fraction}}{2^n}\right). \quad (3)$$

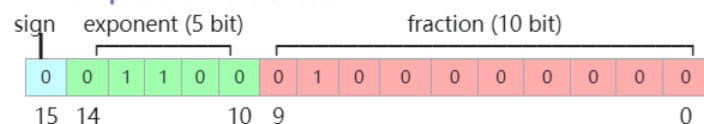
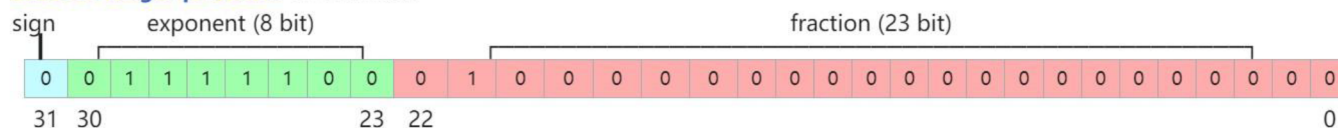


Figure 1—IEEE 754 floating-point representations. The top diagram shows a 32-bit single-precision float, consisting of 1 sign bit, 8 exponent bits (with a bias of 127), and 23 fraction (mantissa) bits. The bottom diagram shows a 16-bit half-precision float, composed of 1 sign bit, 5 exponent bits (with a bias of 15), and 10 fraction bits.

For FP32, the exponent uses 8 bits with a bias of 127, and the fraction (mantissa) has 23 bits, meaning the smallest representable step is 2^{-23} . In contrast, FP16 uses only 5 bits for the exponent (bias = 15) and 10 bits for the fraction, resulting in a coarser resolution of 2^{-10} . Consequently, FP32 supports a much wider dynamic range (approximately 10^{38}) and higher precision (about 7 decimal digits), while FP16 is limited to roughly 10^5 in magnitude and around 3 decimal digits of accuracy. These differences make FP32 more suitable for general scientific and engineering computations, while FP16 is favored in AI and GPU-accelerated workloads where reduced precision is acceptable in exchange for faster computation and lower memory usage.

In this work, we adopt the IEEE FP16 (half-precision floating-point) format to reduce the memory and I/O burden associated with wavefield storage in RTM. Instead of storing the forward wavefield in standard FP32 precision, we cast it to FP16 format during the forward modeling stage. This strategy effectively compresses the wavefield data in-place, reducing the memory footprint by 50% without invoking any additional compression algorithm. The compressed FP16 wavefield is then transferred from GPU memory to host (CPU) memory and optionally written to disk for reconstruction during backward propagation. By using FP16 both as a storage format and as a lightweight compression mechanism, we reduce the cost of device-to-host memory transfers and disk I/O, which are among the most significant performance bottlenecks in large-scale RTM. Importantly, this approach does not alter the numerical computation of the wave equation itself, which remains in FP32 to preserve stability and accuracy during wavefield propagation.

Synthetic data tests

3D impulse response tests

To evaluate the feasibility of using FP16 for wavefield storage, we first performed a series of 3D impulse response tests using three different media: isotropic, vertical transverse isotropy (VTI), and tilted transverse isotropy (TTI). In each test, a single source was placed at the center of the model, and RTM was executed using conventional FP32 precision as a baseline, followed by a second run where the forward wavefield was stored in FP16 format. All numerical computations, including forward and backward wavefield propagation, were kept in FP32 to ensure that any differences in the output could be attributed solely to the reduced-precision wavefield storage and I/O.

Figure 2 displays the resulting migrated impulse responses for all three media types. The key structural features, such as primary reflections and wavefront geometry, are nearly identical between the FP32 and FP16 results. Differences, where present, are minor and appear primarily in low-amplitude regions, often near the edges or noise-level zones. These differences do not degrade the image interpretability or resolution in any meaningful way. This test confirms that FP16 storage can be reliably used for RTM wavefield data without introducing significant artifacts, even in complex anisotropic conditions. It further establishes FP16 as a viable strategy for memory and I/O optimization in high-resolution production RTM workflows.

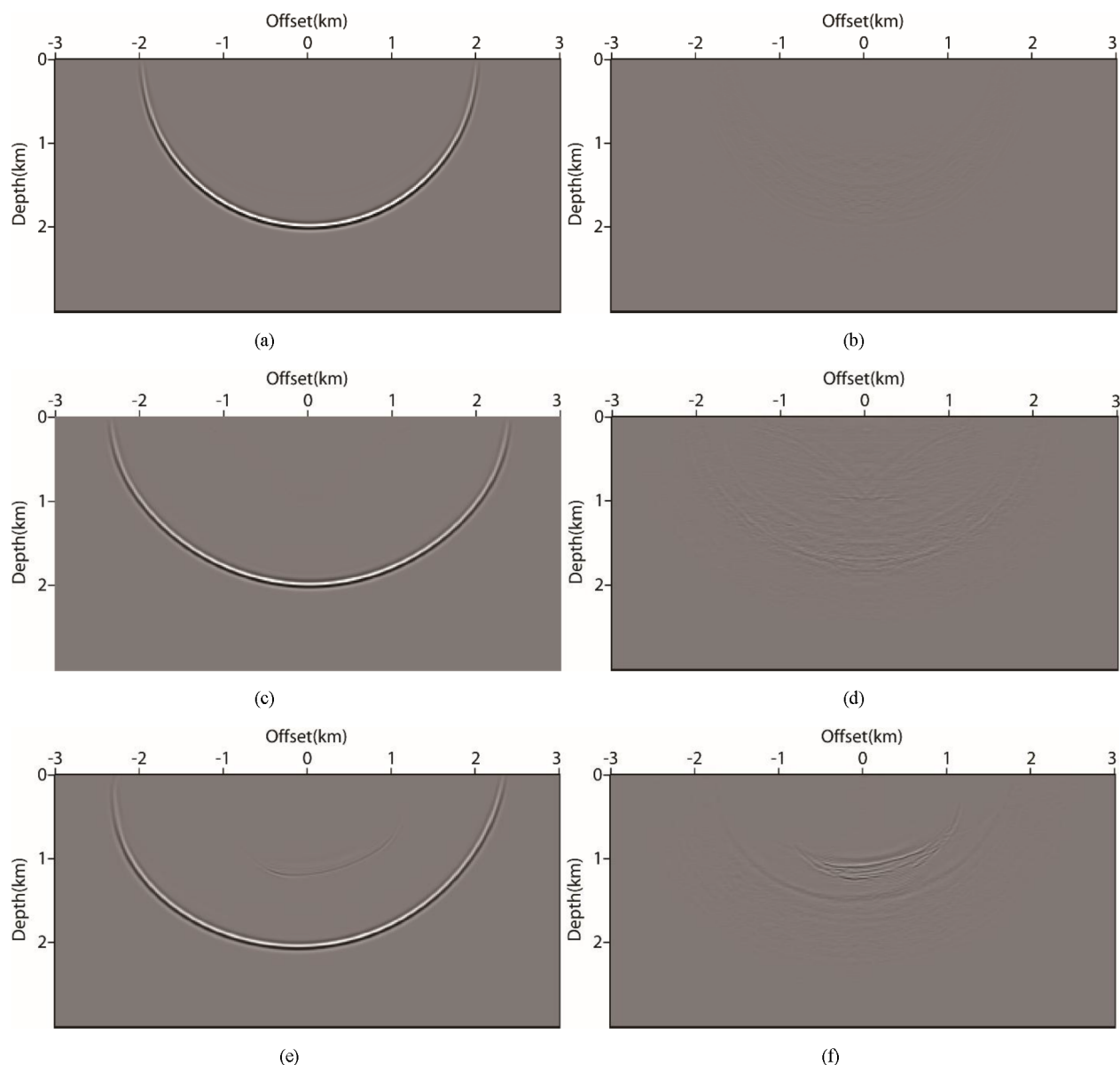


Figure 2—Impulse response for isotropic (a), VTI (c), and TTI (e) media, Figures (b), (d), and (f) are the differences between FP32 and FP16 IO cases, the amplitudes are amplified by a factor 10.

SEG 3D Salt model

To further validate the effectiveness of FP16 wavefield storage in a more realistic setting, we conducted RTM experiments using the SEG/EAGE 3D salt model. This model features complex velocity variations and sharp contrasts, making it a challenging benchmark for seismic imaging. During forward modeling, the wavefield was stored in either FP32 or FP16 precision, while all computations were carried out in FP32. Figure 3 shows representative snapshots of the forward wavefield for both FP32 and FP16 cases. The wavefronts, amplitudes, and propagation patterns exhibit excellent agreement, with only minor differences observed in low-amplitude regions.

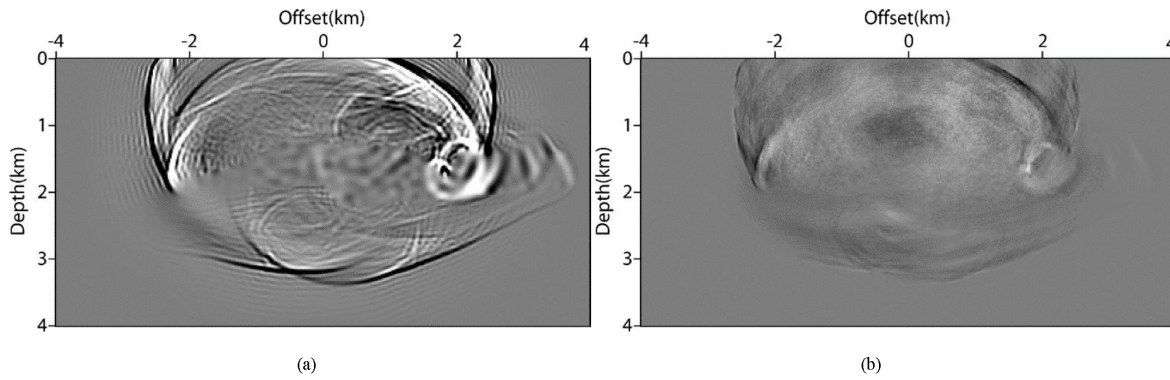


Figure 3—Forward modeling snapshot (a), and (b) are the differences between FP32 and FP16 IO, the amplitude is amplified by a factor 10.

Figure 4 presents the corresponding RTM images and their absolute differences. The imaging results from FP16 and FP32 are visually indistinguishable, capturing the salt body geometry, subsalt reflectors, and structural continuity with high fidelity. The difference panel shows that discrepancies are limited to isolated low-amplitude regions and do not affect the interpretability or resolution of the final image. These results further confirm that FP16 can be safely adopted for wavefield storage in production RTM, offering substantial reductions in memory and I/O without compromising image quality, even in geologically complex models.

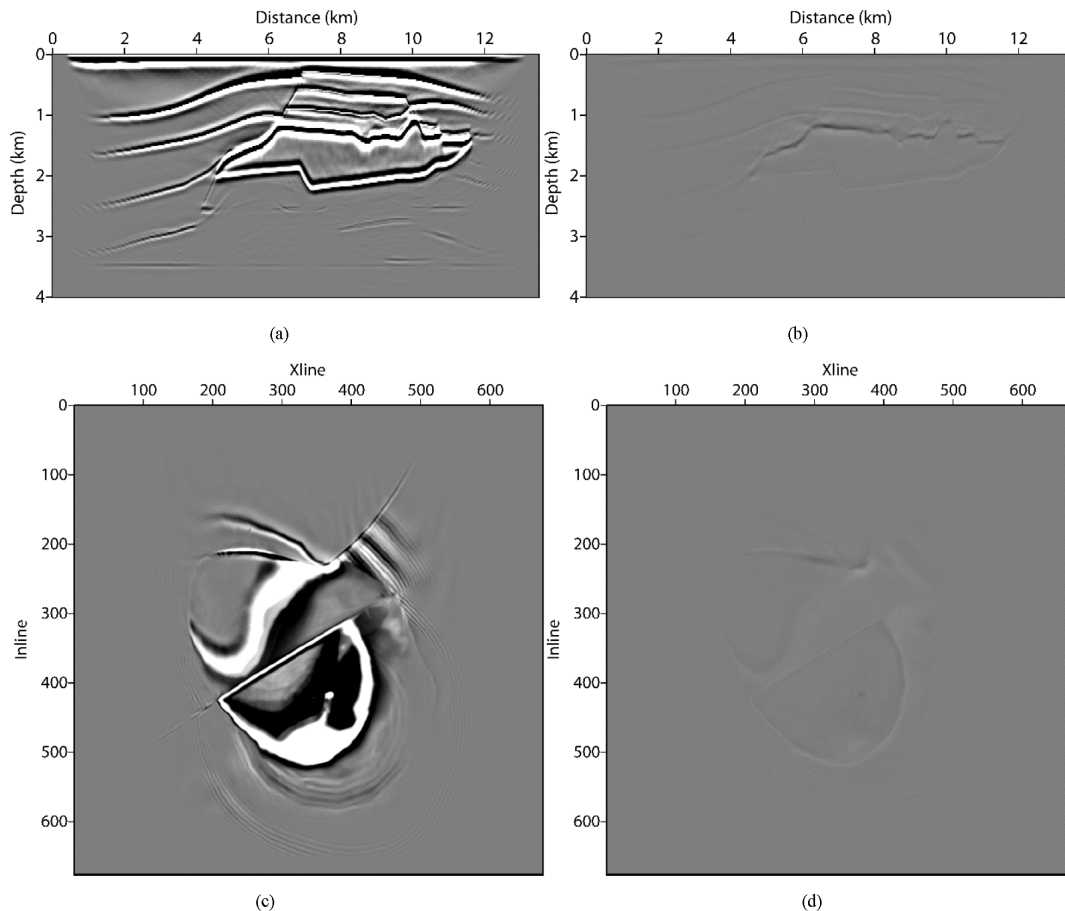


Figure 4—RTM result from 250 shots using only one GPU card. Figures (a) and (c) are RTM inline and depth slice at z=2000m (b) and (d) are the differences between FP32 and FP16 IO, the amplitude is amplified by a factor 10.

Field data tests

To evaluate the practical applicability of FP16 wavefield storage in real-world scenarios, we performed a production-scale RTM test using a field dataset comprising 779 shots. The entire migration was executed on a single NVIDIA RTX 4090 GPU, highlighting the computational efficiency and resource savings enabled by the proposed method. Figure 5 presents the inline and depth slice results from the RTM output using FP16 I/O, along with the corresponding differences from the FP32 baseline. The migrated image accurately captures the subsurface structure, with clear reflectors and consistent amplitudes throughout the section. The difference panels, amplified by a factor of 10 for visibility, show minimal deviation between the FP16 and FP32 results, mostly concentrated in low-amplitude areas that do not affect overall image quality. These results demonstrate that FP16 wavefield storage not only preserves imaging accuracy for field-scale RTM, but also significantly reduces memory and disk requirements, making it a viable solution for high-resolution seismic imaging on commodity GPU hardware.

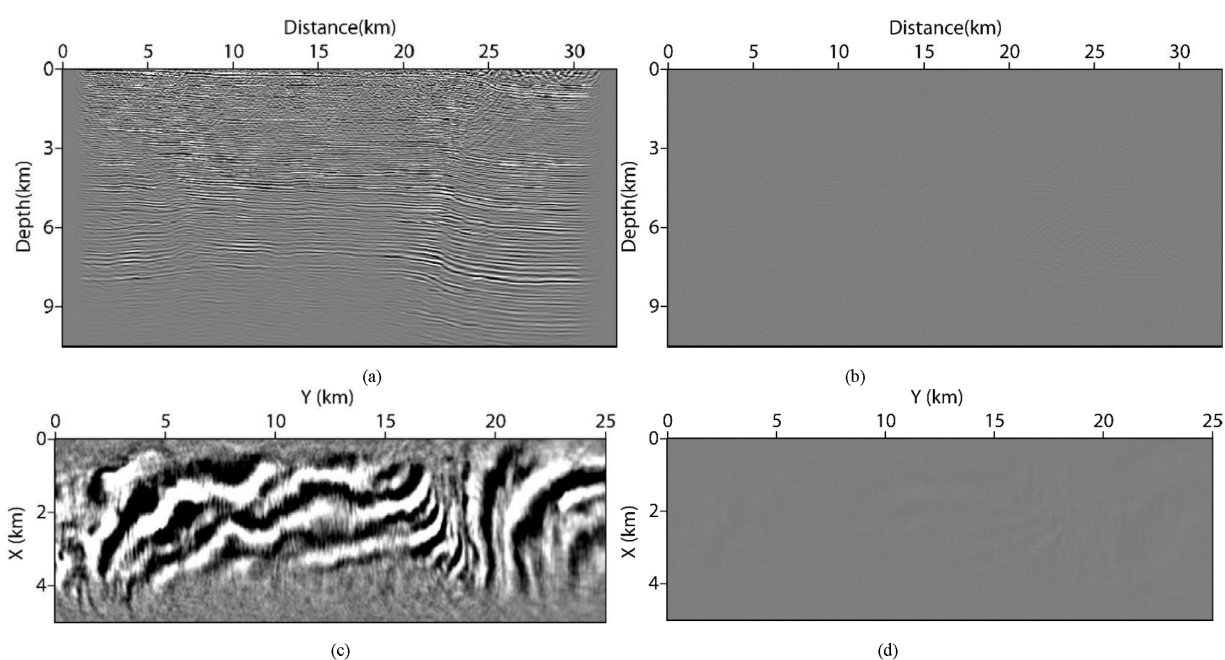


Figure 5—RTM result from 779 shots using only one GPU card. Figures (a) and (c) are RTM inline and depth slice at $z=3500\text{m}$ (b) and (d) are the differences between FP32 and FP16 IO, the amplitude is amplified by a factor 10.

The bar chart in Fig. 6 compares the performance of FP32 and FP16 precision for isotropic (ISO), VTI, and TTI media in terms of runtime, memory usage, and disk I/O. The values plotted are the ratios of FP32 over FP16. While the runtime improvement of FP16 is moderate (around 20–30%), the memory and disk usage are reduced by more than a factor of 2 across all cases. This demonstrates that using FP16 significantly lowers the data footprint, which is especially advantageous in large-scale wave-equation simulations where memory bandwidth and storage throughput are critical constraints. The results highlight FP16's potential as a practical solution for reducing resource consumption in high-performance numerical modeling.

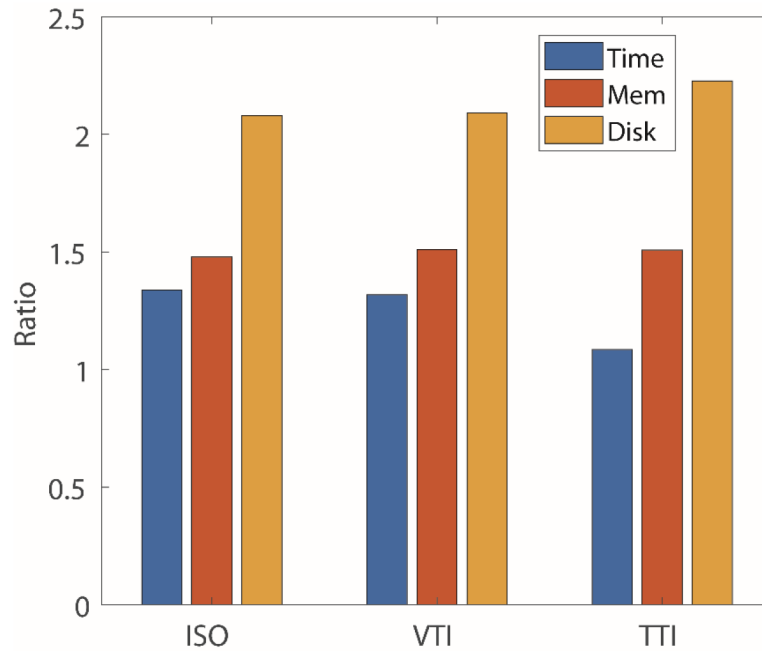


Figure 6—Performance comparison between FP32 and FP16 implementations for isotropic (ISO), VTI, and TTI media.

Discussion

FP16 format for computation?

To investigate whether FP16 can also be used for wavefield computation, we repeated the 3D impulse response tests for isotropic, VTI, and TTI models using FP16 not only for storage but also for forward and backward wavefield propagation. Figure 7a, 7c, and 7e show the resulting RTM images for the three media types, while Figures 7b, 7d, and 7f present the corresponding differences from the FP32 baseline. In contrast to the previous results where FP16 was used only for I/O, the use of FP16 for computation introduces noticeable degradation in the image quality. The differences are no longer confined to low-amplitude noise but appear as clear phase shifts, amplitude distortion, and defocusing in the main reflection events.

These artifacts are particularly evident in the TTI case, where the imaging condition is more sensitive to numerical precision due to the complexity of wave propagation and anisotropy parameters. The differences shown in Figures 7b, 7d, and 7f highlight that using FP16 for full wavefield computation introduces unacceptable errors, compromising the reliability of the RTM results.

Based on these tests, we conclude that current IEEE FP16 precision is not suitable for production-level RTM computation, especially in complex media. The loss of numerical resolution in wavefield updates leads to unstable propagation and degraded imaging. Therefore, while FP16 is effective for wavefield storage and I/O compression, high-fidelity RTM still requires full FP32 precision for computation to ensure stable and accurate imaging results.

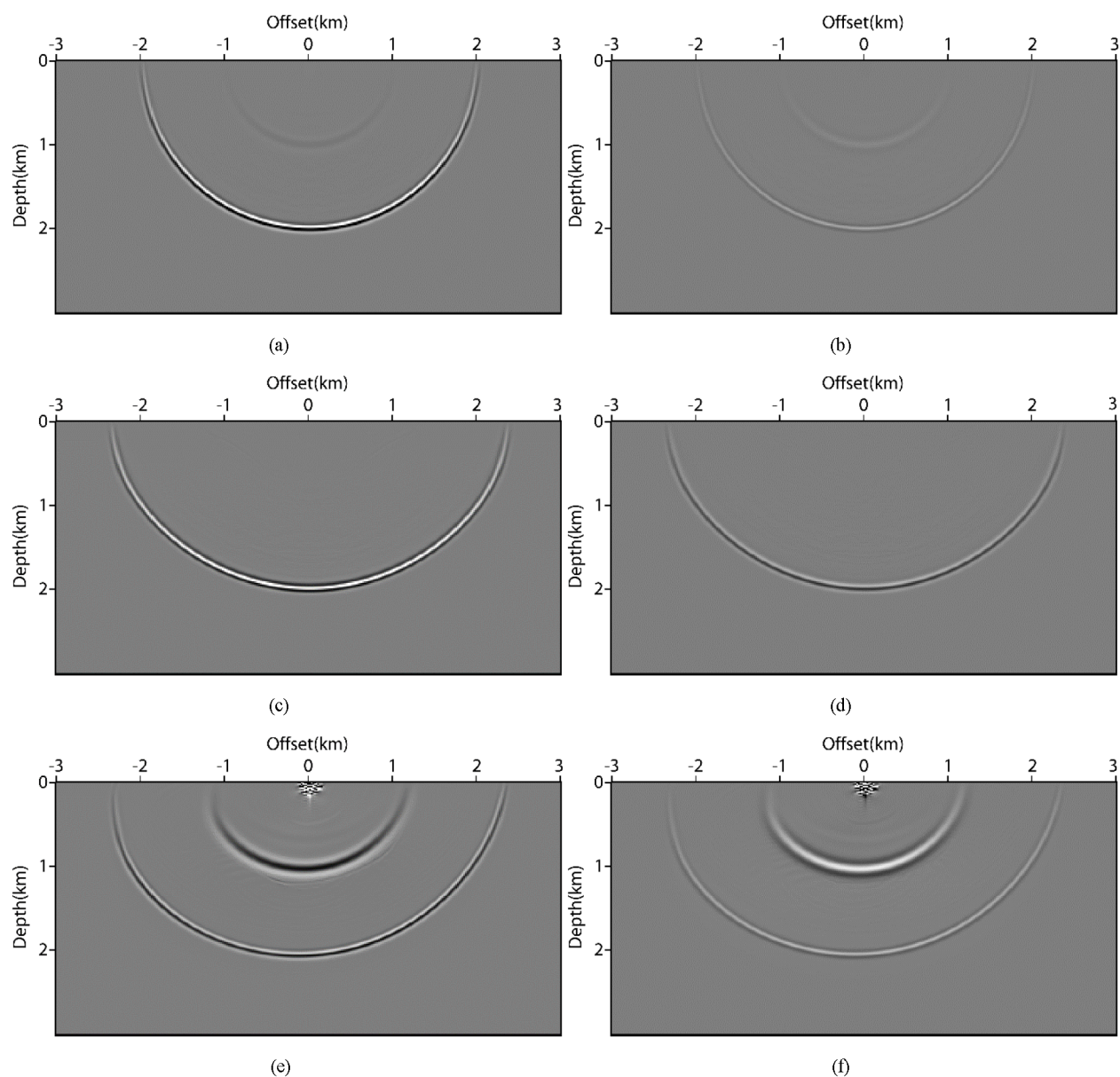


Figure 7—Impulse response for isotropic (a), VTI (c), and TTI (e) media, (b), (d), and (f) are the differences between FP32 and FP16, the amplitude is not amplified.

Multi-job on one GPU card

To explore the practical benefit of FP16 in maximizing hardware utilization, we conducted a multi-instance scalability test by submitting multiple concurrent RTM jobs on a single GPU card. Figure 8 summarizes the results for isotropic (ISO), VTI, and TTI models using both FP32 and FP16 precision. In this test, each RTM job runs independently, and the GPU's ability to handle multiple jobs in parallel is evaluated based on total runtime and relative speedup.

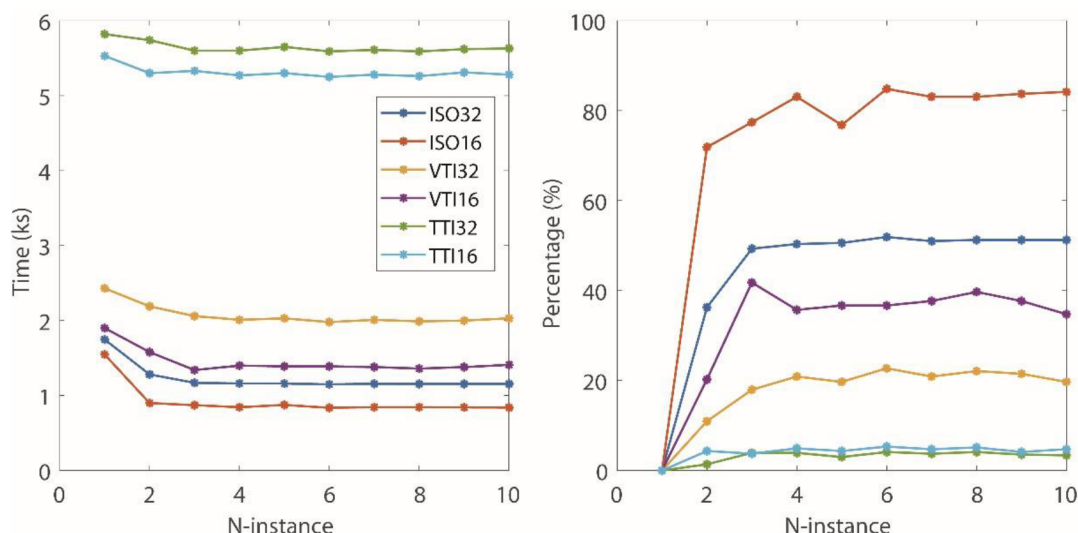


Figure 8—Runtime and FP16 speedup when submitting multiple jobs per GPU. Left: Total runtime vs. number of concurrent jobs. Right: FP16 speedup (%) over FP32. Initial speedup is significant with 2 jobs, but further scaling shows diminishing returns.

As shown in the left panel of Figure 8, submitting two concurrent jobs per GPU leads to a noticeable reduction in total execution time across all models, with FP16 configurations consistently outperforming their FP32 counterparts. This is largely due to FP16's reduced memory footprint, which allows more data to fit into GPU memory, minimizing the need for memory swapping and reducing device-host transfers. ISO16, in particular, achieves the most significant runtime reduction when moving from one to two instances.

However, as the number of concurrent jobs increases beyond two, the overall runtime begins to plateau. This behavior is more pronounced in the FP32 cases but is also observed with FP16. The right panel of Figure 8 illustrates the corresponding speedup percentages of FP16 over FP32. The speedup peaks at around 80–90% for the isotropic case with two instances, but shows diminishing returns with more concurrent jobs. This plateauing trend indicates saturation of critical GPU resources such as shared memory, register files, or streaming multiprocessor (SM) occupancy. It also suggests that kernel-level parallelism, although efficient for one or two jobs, becomes bottlenecked when multiple jobs compete for the same low-level resources.

These results highlight a key advantage of FP16: by reducing memory and I/O overhead, it enables better exploitation of multi-instance concurrency on a single GPU. However, the benefit is bounded by hardware constraints. Thus, in large-scale RTM deployments, combining FP16-based I/O optimization with task-level parallelism across multiple GPUs or nodes may provide a more scalable path to performance gains.

Conclusions

In this study, we have demonstrated the effectiveness of using FP16 (half-precision floating-point) format for optimizing memory and I/O efficiency in reverse time migration (RTM) workflows on GPUs. By converting the forward wavefield to FP16 format during storage and transfer—while maintaining FP32 precision for wavefield computations—we achieved significant reductions in GPU memory usage and disk I/O, without compromising image quality. Through a series of 3D impulse response tests, SEG salt model experiments, and field data migrations, we showed that the differences between FP32 and FP16 outputs are negligible, confirming the numerical stability and accuracy of the FP16-based I/O strategy.

Further, we demonstrated that reduced memory usage enabled by FP16 allows for the submission of multiple concurrent RTM jobs on a single GPU, improving hardware utilization. Notably, substantial performance gains were observed when running two simultaneous jobs, although diminishing returns appeared beyond that point due to hardware resource saturation. These findings highlight the practical value of FP16 in real-world seismic imaging scenarios, particularly for high-throughput production environments.

However, we also evaluated the use of FP16 for wavefield computation and found that it introduces significant numerical errors, especially in complex anisotropic media. These errors manifest as amplitude distortions and phase shifts that are unacceptable for production-level imaging. As such, current IEEE FP16 precision is not recommended for full RTM computation.

Looking ahead, future research could explore hybrid precision strategies, such as dynamic precision control or block-wise adaptive encoding, to selectively apply FP16 in less sensitive parts of the domain. Emerging GPU hardware with support for bfloat16 or higher dynamic range FP8 formats may also offer new opportunities to balance performance with accuracy. Additionally, combining FP16-based I/O optimization with distributed GPU clusters and advanced job schedulers may further improve throughput for industrial-scale seismic imaging.

References

- Deng, F., Z. Zhang, and Y. Liu, 2021, Data mapping strategies for multi-GPU implementation of a seismic application: *Proceedings of the 2021 SBAC-PAD Symposium*, <https://doi.org/10.1109/SBAC-PAD53660.2021.00047>.
- Ginsburg, B., A. Lavin, P. Micikevicius, and G. Venkatesh, 2020, Accelerating deep learning workloads through mixed precision: NVIDIA Developer Blog, <https://developer.nvidia.com/blog/mixed-precision-training-deep-neural-networks/>.
- Jia, Y., E. Shelhamer, R. B. Girshick, S. Zhang, S. Guadarrama, and T. Darrell, 2019, Highly scalable deep learning training system with mixed precision: Training ImageNet in four minutes: arXiv preprint, arXiv: 1811.05233. <https://arxiv.org/abs/1811.05233>.
- Jouppi, N. P., C. Young, N. Patil, et al., 2017, In-datacenter performance analysis of a tensor processing unit: *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, IEEE, Toronto, <https://doi.org/10.1109/ISCA.2017.7985637>.
- Kronenberg, J., W. Engelke, and W. A. Schneider, 2007, GPUs: An efficient platform for Kirchhoff migration: *Geophysics*, **72**(5), S29–S34, <https://doi.org/10.1190/1.2761840>.
- Lin, F., S. Lin, L. Wang, and Z. Xu, 2022, Applications of multi-instance GPUs (MIG) for geophysical simulations: *Computational Geosciences*, **26**, 1371–1384, <https://doi.org/10.1007/s10596-022-10125-8>.
- Liu, G. F., X. H. Meng, Z. J. Yu, and D. J. Liu, 2019, An efficient scheme for multi-GPU TTI reverse time migration: *Applied Geophysics*, **16**(1), 56–63, <https://doi.org/10.1007/s11770-019-0728-8>.
- Liu H., B. Li, H. Liu, X. Tong, Q. Liu, 2010, The algorithm of high order finite difference pre-stack reverse time migration and GPU implementation: *Chinese Journal of Geophysics*, **53**(7): 1725–1733, doi: 10.1002/cjg2.1530.
- Liu H., Y. Luo, 2020, Making true amplitude angle domain common image gathers using invertible Radon transform, *Communications in Computational Physics*, **28**, 128–140, doi: 10.4208/cicp.OA-2018-0074.
- Liu H., B. Li, H. Liu, X. Tong, Q. Liu, 2012, The issues of pre-stack reverse time migration and solutions with GPU implementation. *Geophysical Prospecting*, **60**, 906–918, doi: 10.1111/j.1365-2478.2011.01032.x.
- Liu H., R. Ding, L. Liu and H. Liu, 2013, Wavefield reconstruction methods for reverse time migration: *Journal of Geophysics and Engineering*, **10**(1), 015004, doi: 10.1088/1742-2132/10/1/015004.
- Micikevicius, P., 2009, 3D finite difference computation on GPUs using CUDA: Proceedings of the 2nd Workshop on General-Purpose Processing on Graphics Processing Units, ACM, New York, <https://doi.org/10.1145/1639642.1639657>.
- Micikevicius, P., S. Narang, J. Alben, et al., 2018, Mixed precision training: *Proceedings of the International Conference on Learning Representations (ICLR)*, <https://arxiv.org/abs/1710.03740>.
- Modave, A., A. St-Cyr, W. A. Mulder, and T. Warburton, 2015, Nodal discontinuous Galerkin simulations for reverse-time migration on GPU clusters: arXiv preprint, arXiv:1506.00907, <https://arxiv.org/abs/1506.00907>.
- Moussa, N. W., and G. T. Schuster, 2015, Seismic imaging using GPGPU accelerated reverse time migration: *Stanford Exploration Project*, **138**, 1–12, <http://sepwww.stanford.edu/data/media/public/docs/sep138/nwmoussa1/paper.pdf>.
- NVIDIA Corporation, 2021, Mixed precision training guide: NVIDIA Developer Documentation, <https://docs.nvidia.com/deeplearning/performance/mixed-precision-training/>.
- Peng, Z., S. Yuan, X. Qiu, W. Zhang, and A. Sowizdzal, 2023, Advances and applications of artificial intelligence in geoscience and remote sensing: *Frontiers in Earth Science*, **11**, 1234360, <https://doi.org/10.3389/feart.2023.1234360>.
- PyTorch Team, 2020, What every PyTorch user should know about mixed precision training: PyTorch Blog, <https://pytorch.org/blog/what-every-user-should-know-about-mixed-precision-training-in-pytorch/>.
- Smith, T., and J. Klein, 2023, Seismic data processing in the age of AI: Challenges and opportunities: *Geophysical Prospecting*, **71**(4), 635–647, <https://doi.org/10.1111/1365-2478.13105>.

- Yin, T., and Y. Zhang, 2020, Half-precision floating point applications in wave-equation-based seismic inversion: *SEG Technical Program Expanded Abstracts 2020*, 4580–4584, <https://doi.org/10.1190/segam2020-3423532.1>.
- Yu, S., and J. Ma, 2021, Deep learning for geophysics: Current and future trends: *Reviews of Geophysics*, **59**(3), e2021RG000742, <https://doi.org/10.1029/2021RG000742>.
- Zhang, W., X. Sun, W. Wang, and C. Tang, 2019, Accelerating large-scale seismic wavefield computations on GPUs using FP16 mixed precision arithmetic: *Journal of Applied Geophysics*, **169**, 98–108, <https://doi.org/10.1016/j.jappgeo.2019.07.005>.
- Zhu, Z., J. Yan, and H. Liu, 2021, Enhancing computational efficiency in 3-D seismic modelling with half-precision floating-point format: *Geophysical Journal International*, **238**(3), 1595–1607, <https://doi.org/10.1093/gji/ggaa576>.